WBS TRAINING

Funktionales Interface Lambda

OP2 - Die Säulen der Objektorientierten Programmierung

Java

WBS TRAINING AG
Lorenzweg 5
D-12099 Berlin
Amtsgericht Berlin HRB 68531
Sitz der Gesellschaft: Berlin

Vorstand:
Heinrich Kronbichler,
Joachim Giese
Aufsichtsrat (Vorsitz): Dr. Daniel Stadler
USt-IdNr.: DE 209 768 248

GLS Gemeinschaftsbank eG
IBAN: DE18 4306 0967 1146 1814 00
BIC: GENODEM1GLS

DQS-zertifiziert nach
DIN EN ISO 9001:2015 Reg.-Nr. 015344 QM15
Zulassung nach AZAV Reg.-Nr. 015344 AZAV

WBS TRAINING

Inhalt

1 Funktionales Interface und Lambda 2

1.1 Begriffe 2

1.1.1 Functional Interface 2

1.1.2 Lambda 2

1.2 Aufbau Lambda 3

1.2.1 Komplexere Lambda-Ausdrücke 3

1.2.2 Lambda-Ausdrücke ohne Parameter 4

1.3 Typische Anwendungsfälle 4

1.3.1 Lambda-Ausdrücke als Argumente an Methoden 4

1.3.2 Iterieren 5

1.3.3 Consumer Interface 5

1.3.4 Weitere Interfaces für Lambdas 6

1.4 UML 7

1 Funktionales Interface und Lambda

1.1 Begriffe

1.1.1 Functional Interface

Ein **Functional Interface** ist ein Interface, welches **nur eine abstrakte Methode** bereitstellt. Dies bedeutet, dass ein funktionales Interface nur 1 Verhalten repräsentiert. Ein funktionales Interface kann aber zusätzlich zur abstrakten Methode beliebig viele *default* und statische Methoden bereitstellen. Ein funktionales Interface kann andere Interfaces erweitern, solange diese keine abstrakten Methoden beinhalten. Häufig mit der Annotation *@FunctionalInterface* versehen.

```
@FunctionalInterface
interface Rechner {
    int berechne(int a, int b); //implizit abstract
}
```

1.1.2 Lambda

Ein Functional Interface ist die Voraussetzung dafür, dass du einen **Lambda**-Ausdruck verwenden darfst. Oder andersherum: Lambda-Ausdrücke sind **eine kompakte Art, ein Functional Interface zu implementieren**. Lambdas sind **Ausdrücke**, keine Klassen. Lambda Ausdrücke ähneln Methoden, sie haben Parameter, einen Körper und einen Rückgabewert. Man kann sie auch als anonyme Methoden bezeichnen.

Implementierung umständlich als anonyme Klasse:

```
Rechner addierer = new Rechner() {
    @Override
    public int berechne(int a, int b) {
        return a + b;
    }
};
```

Implementierung besser als Lambda: Java *weiß* durch den Kontext (z. B. Variable Rechner), **welches Functional Interface** implementiert wird.

```
Rechner addierer = (a, b) -> a + b;
addierer.berechne(2,3);
```

1.2 Aufbau Lambda

```
int berechne(int a, int b); //aus Interface

Rechner addierer = (a, b) -> a + b; //Lambda
```

Parameter und Rückgabe müssen der abstrakten Methode des Interfaces entsprechen. Der Pfeil verbindet die Parameterliste mit dem Funktionskörper.
Syntax in kurz: (Parameterliste) -> { Funktionskörper }

1.2.1 Komplexere Lambda-Ausdrücke

Lambda-Ausdrücke können beliebig viele Anweisungen im Methodenkörper haben. In diesen Fällen muss der Körper aber mit geschwungen Klammern angegeben werden. Werden geschwungene Klammern für den Funktionskörper verwendet, müssen wir für die Rückgabe das *return*-Keyword angeben. Wenn kein Datentyp bei Parametern angegeben wurde, dann nimmt sich der Lambda-Ausdruck den Typ aus dem Kontext. Wir können aber die Datentypen zur besseren Lesbarkeit trotzdem angeben:

```
@FunctionalInterface
public interface QuadratZahl
{
   public abstract int berechneQuadratZahl(int zahl);
}


QuadratZahl quad = (int zahl) ->
{
   System.out.println("Übergebener Wert: " + zahl);
   return zahl * zahl;

};

ergebnisLambda = quad.berechneQuadratZahl(5);
System.out.println("Das Ergebnis ist: " + ergebnisLambda);
```

1.2.2 Lambda-Ausdrücke ohne Parameter

Hat die Funktion/Methode keine Parameter, müssen trotzdem runde Klammern angegeben werden.

```
@FunctionalInterface
public interface Printable
{
   public abstract void print();
}

Printable halloWelt = () -> System.out.println("Hallo, Welt!");
halloWelt. print ();
```

1.3 Typische Anwendungsfälle

1.3.1 Lambda-Ausdrücke als Argumente an Methoden

Lambda-Ausdrücke können als Argumente an Methoden übergeben werden. Dies ist auch eine der Hauptzwecke von Lambda-Ausdrücken. Lambda ermöglicht es uns also, Funktionalität an Methoden zu übergeben. Funktionalität wird wie Daten behandelt. Dieses Verfahren wird z.B. häufig in der GUI Entwicklung eingesetzt.

```
@FunctionalInterface
public interface QuadratZahl
{
   public abstract int berechneQuadratZahl(int zahl);
}

In Klasse:

...

private static int berechne(QuadratZahl lambda, int i)
{
   return lambda.berechneQuadratZahl(i);
}


ergebnis = berechne((a) -> a * a, 9);
```

1.3.2 Iterieren

Übergabe Lambda an *foreach* Methode von Listen, anstatt Ausgabe über *foreach*-Schleife:

```
ArrayList<String> tiere = new ArrayList<>();
tiere.add("Krokodil");
tiere.add("Nilpferd");
tiere.add("Elefant");

tiere.forEach(tier -> System.out.println(tier));
```

Sonderfall: die *println()*-Methode hat eine Signatur, die identisch mit dem *Consumer*-Interface ist (*void* mit String-Parameter). Darum können wir hier verkürzt eine Methodenreferenz anstelle der Lambda-Expression angeben. Anstatt des .-Operators wird dann :: verwendet:

```
tiere.forEach(System.out::println);
```

1.3.3 Consumer Interface

Das Consumer<T>-Interface ist ein **vordefiniertes funktionales Interface** aus dem Paket java.util.function. Es wird verwendet, wenn du eine Operation auf ein Objekt vom Typ T ausführen willst, **ohne etwas zurückzugeben**. Es hat **eine einzige abstrakte Methode***: void accept(T t)* und eine *default*-Methode *andThen(T t)*. **Rückgabewert:** *void* → es wird **etwas getan**, aber nichts zurückgegeben (z. B. ausgeben, speichern, senden …)

```
//Lambda-Expression zuweisen, oder eine passende Methodenreferenz

Consumer<String> out = System.out::println;

// Mit 'andThen()' können wir die Methoden verketten:
out = out.andThen(s -> System.out.println(s.toUpperCase()));
oot = out.andThen(s -> System.out.println(s.toLowerCase()));

// Mit 'accept()' führen wir alle gespeicherten Anweisungen aus:
out.accept("Hallo, Welt!");
```

1.3.4 Weitere Interfaces für Lambdas

Function Interface (Erster Typ für den Parameter, zweiter Typ für die Rückgabe)

```
Function<Integer, Integer> quadrat = (Integer x) -> x * x;
System.out.println(quadrat.apply(10));
```

Predicate Interface (Einen Parameter, gibt immer Boolean zurück)

```
Predicate<Integer> groeßer10 = (x) -> x > 10;
System.out.println(groeßer10.test(5));
```

Comparator Interface (Zwei Parameter vom selben Typ und gibt immer int zurück)

```
Comparator<String> vergleich =
    (String x, String y) -> Integer.compare(y.length(), x.length());
System.out.println(vergleich.compare("Hallo", "Welt"));

Comparator<Integer> ivergleich = Integer::compare;
System.out.println(ivergleich.compare(2,3));
```

1.4 UML

Functional Interfaces in UML:

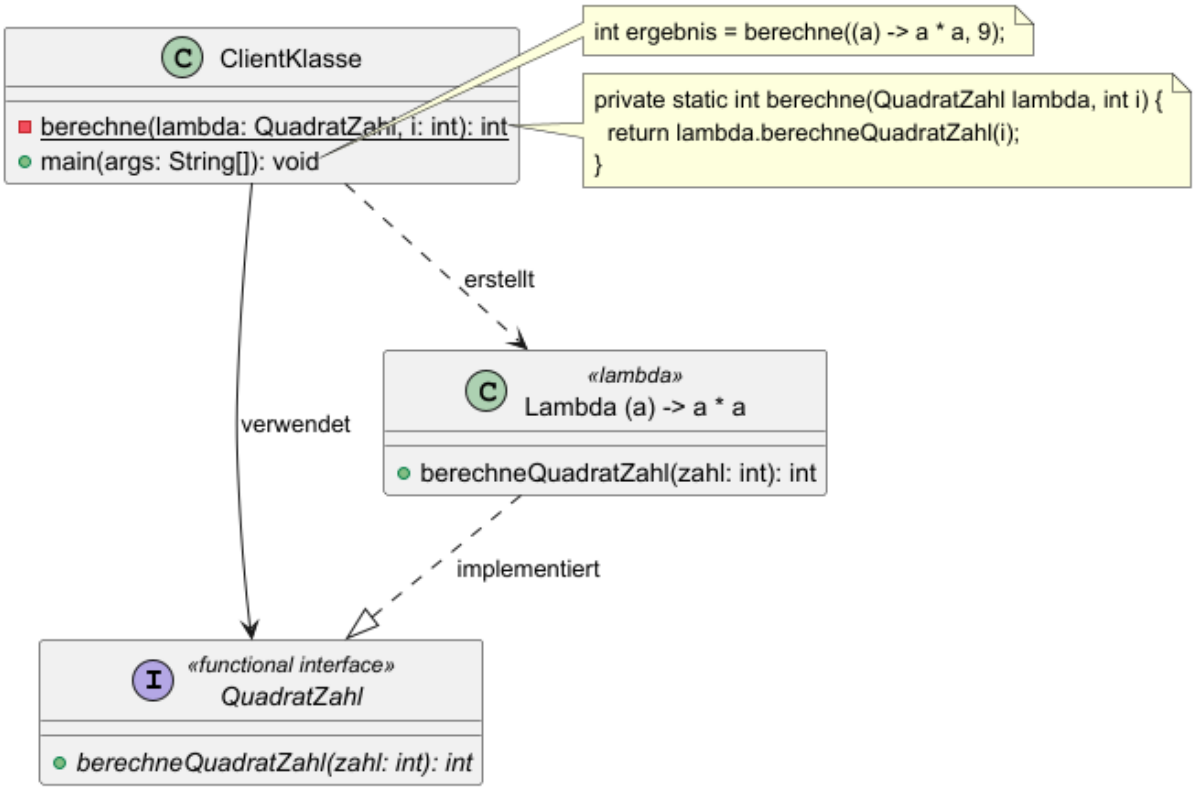
- Ein Functional Interface wird als normales Interface mit einer einzelnen abstrakten Methode dargestellt
- Es kann mit einem Stereotyp *«functional interface»* gekennzeichnet werden, um es als funktionales Interface zu kennzeichnen

Lambda-Ausdrücke in UML:

- UML hat keine standardisierte Notation speziell für Lambda-Ausdrücke
- Übliche Darstellungsmöglichkeiten:
    - Als anonyme Klasse
    - Mit einem speziellen Stereotyp *«lambda»* für eine anonyme Klasse
    - Als Notiz oder Kommentar zum Kontext, in dem das Lambda verwendet wird

Beziehungen:

- Die implementierende Beziehung zwischen Lambda und Functional Interface wird meist wie eine normale Interface-Implementierung dargestellt
- Der Fokus liegt eher auf dem Kontext der Lambda-Nutzung


ClientKlasse
berechne(lambda: QuadratZahl, i: int): int
main(args: String[]): void
int ergebnis = berechne((a) -> a * a, 9);
private static int berechne(QuadratZahl lambda, int i) {
return lambda.berechneQuadratZahl(i);
}
erstellt
verwendet
«lambda»
Lambda (a) -> a * a
berechneQuadratZahl(zahl: int): int
implementiert
«functional interface»
QuadratZahl
berechneQuadratZahl(zahl: int): int